

CE 206: Engineering Computation Sessional

Ordinary Differential Equations
(ODE)

Why study Differential Equations?

Many physical phenomena are best formulated mathematically in terms of their rate of change.

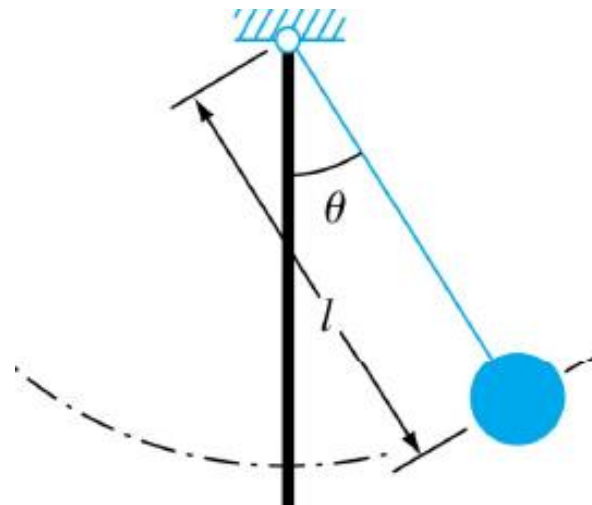


Bungee-jumper Equation

$$\frac{dv}{dt} = g - \frac{c_d}{m} v^2$$

Motion of a swinging pendulum

$$\frac{d^2\theta}{dt^2} + \frac{g}{l} \sin \theta = 0$$



Euler's Method

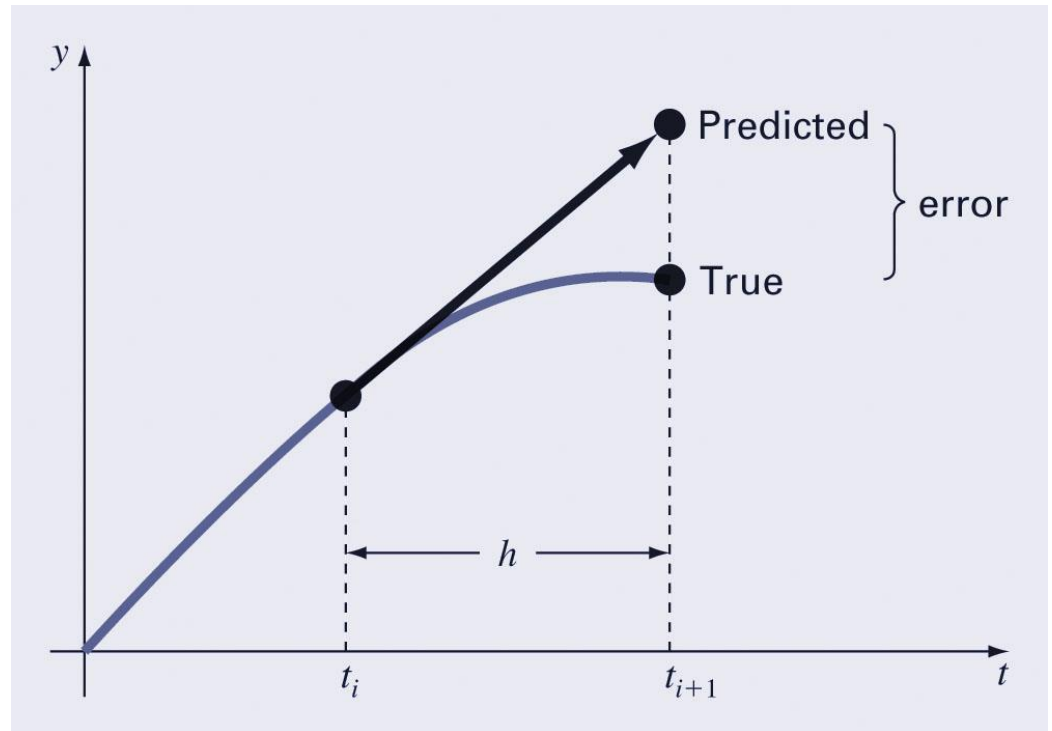
The first derivative provides a direct estimate of the slope at t_i :

$$\left. \frac{dy}{dt} \right|_{t_i} = f(t_i, y_i)$$

Euler method uses that estimate as the increment function:

$$\phi = f(t_i, y_i)$$

$$y_{i+1} = y_i + f(t_i, y_i)h$$



MATLAB Code: Euler's Method

```
function [t,y] = eulode(dydt,tspan,y0,h)
% input: dydt = name of the M-file that evaluates the ODE
% tspan = [ti, tf], ti and tf limits of independent variable
% y0 = initial value of dependent variable    h = step size
% output: y = solution vector

if nargin<3,error('less than 3 input arguments'),end
ti = tspan(1); tf = tspan(2);
t = (ti:h:tf)'; n = length(t);
% if necessary, add an additional value of t
% so that range goes from t = ti to tf
if t(n)<tf
    t(n+1) = tf;    n = n+1;
end
y(1)= y0;
for i = 1:n-1 %implement Euler's method
    y(i+1) = y(i) + dydt(t(i),y(i))*(t(i+1)-t(i));
end
```

$$y_{i+1} = y_i + f(t_i, y_i)h$$



MATLAB Code: Euler's Method

```
function [t,y] = eulode(dydt,tspan,y0,h)
% input: dydt = name of the M-file that evaluates the ODE
% tspan = [ti, tf], ti and tf limits of independent variable
% y0 = initial value of dependent variable    h = step size
% output: y = solution vector

if nargin<3,error('less than 3 input arguments'),end
ti = tspan(1); tf = tspan(2);
t = (ti:h:tf)'; n = length(t);
% if necessary, add an additional value of t
% so that range goes from t = ti to tf
if t(n)<tf
    t(n+1) = tf;    n = n+1;
end
y = y0*ones(n,1); %preallocate y to improve efficiency
for i = 1:n-1 %implement Euler's method
    y(i+1) = y(i) + dydt(t(i),y(i))*(t(i+1)-t(i));
end
```

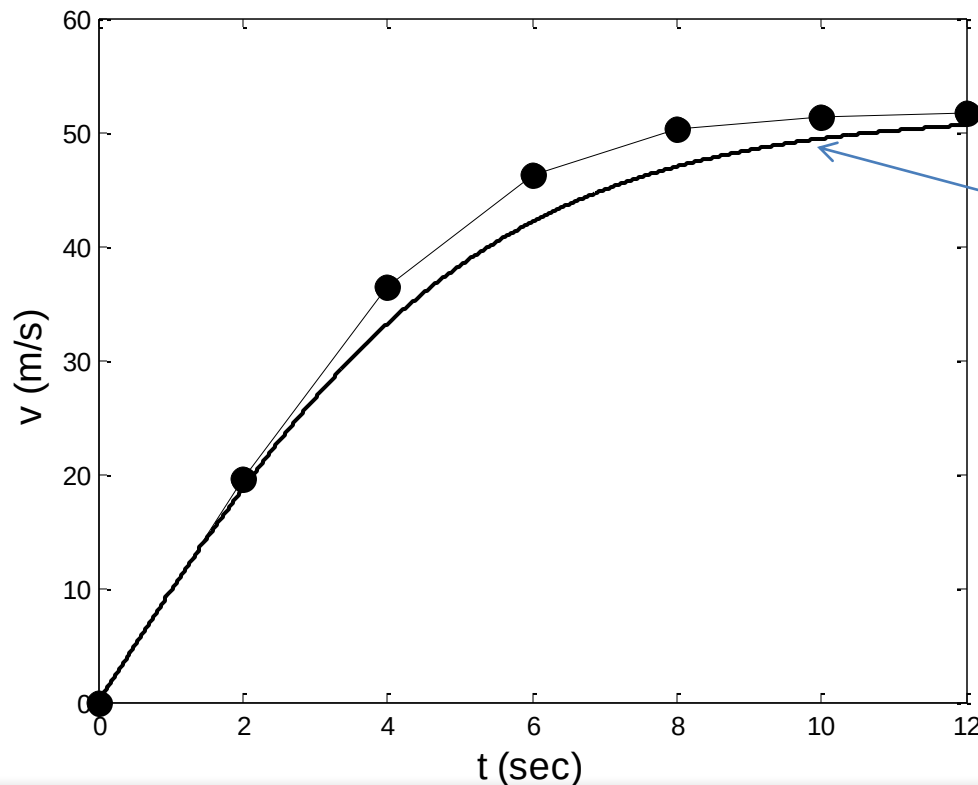
$$y_{i+1} = y_i + f(t_i, y_i)h$$

Exercise: Euler's method

Solve the bungee jumper problem using Euler's method (use a step size of 2 s). Compare the results with the analytical solution

$$\frac{dv}{dt} = g - \frac{c_d}{m} v^2$$

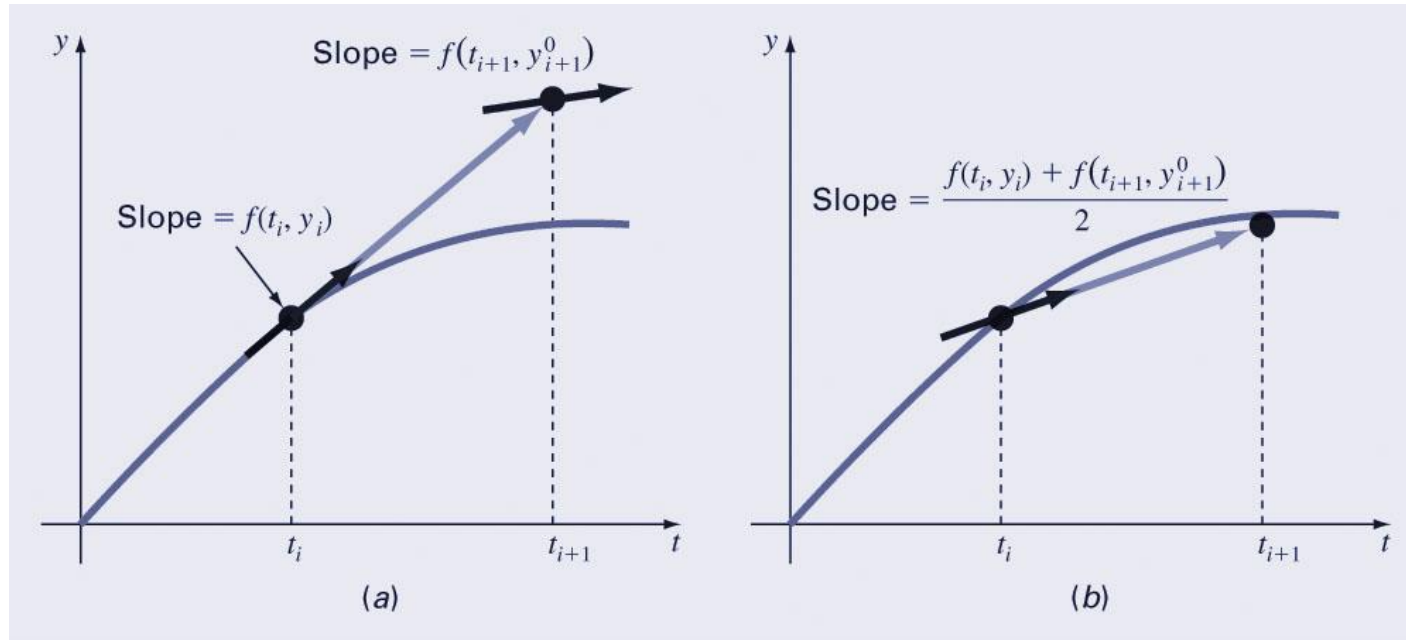
Given, $g = 9.81 \text{ m/s}^2$, $c_d = 0.25 \text{ kg/m}$, $m = 68.1 \text{ kg}$



Analytical solution:

$$v(t) = \sqrt{\frac{gm}{c_d}} \tanh\left(\sqrt{\frac{gc_d}{m}} t\right)$$

Heun's Method



Predictor: $y_{i+1}^0 = y_i + f(t_i, y_i)h$

Corrector: $y_{i+1} = y_i + \frac{f(t_i, y_i) + f(t_{i+1}, y_{i+1}^0)}{2} h$

Exercise: Modify eulode.m to implement Heun's method. Solve the same bungee-jumper problem.

4th order Runge-Kutta method

$$y_{i+1} = y_i + \frac{1}{6} (k_1 + 2k_2 + 2k_3 + k_4)h$$

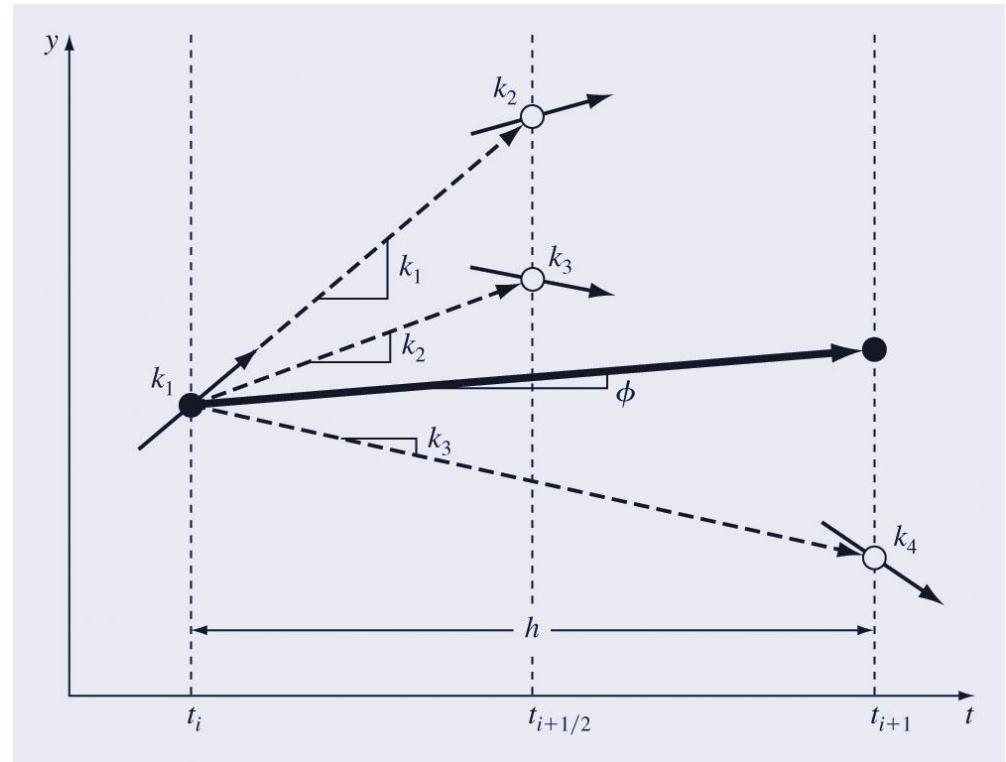
where

$$k_1 = f(t_i, y_i)$$

$$k_2 = f\left(t_i + \frac{1}{2}h, y_i + \frac{1}{2}k_1h\right)$$

$$k_3 = f\left(t_i + \frac{1}{2}h, y_i + \frac{1}{2}k_2h\right)$$

$$k_4 = f(t_i + h, y_i + k_3h)$$



MATLAB Code: Runge-Kutta Method

```
function [tp,yp] = rk4(dydt,tspan,y0,h)
% input:    dydt = name of the M-file that evaluates the ODEs
%    tspan = [ti, tf]; initial and final times
%    y0 = initial values of dependent variables
%    h = step size
% output: tp = vector of independent variable
%    yp = vector of solution for dependent variables

if nargin<3,error('3 input arguments required'), end
ti = tspan(1); tf = tspan(2);
tp = (ti:h:tf)'; n = length(tp);
% if necessary, add an additional value of t
% so that range goes from tp = ti to tf
if tp(n)<tf
    tp(n+1) = tf;    n = n+1;
end
```

MATLAB Code: Runge-Kutta Method

```
yp = y0*ones(n,1); %preallocate y to improve efficiency

for i = 1:n-1 %implement RK method

    tt = tp(i); yy = yp(i);

    k1 = dydt(tt,yy);
    k2 = dydt(tt + h/2, yy + 0.5*k1*h);
    k3 = dydt(tt + h/2, yy + 0.5*k2*h);
    k4 = dydt(tt + h, yy + k3*h);

    yp(i+1) = yy + h*(k1 + 2*k2 + 2*k3 + k4)/6;

end
```

Practice Problem

Solve the following initial value problem over the interval from $t = 0$ to 2 where $y(0) = 1$. Display all your results in the same graph

$$\frac{dy}{dt} = yt^3 - 1.5y$$

- (a) Analytically
- (b) Euler's method with $h = 0.5$ and $h = 0.25$
- (c) Heun's method with $h = 0.5$
- (d) 4th order RK method with $h = 0.5$

MATLAB's common ODE solvers

MATLAB's **ode23** function uses second- and third-order RK functions to solve the ODE and adjust step sizes.

MATLAB's **ode45** function uses fourth- and fifth-order RK functions to solve the ODE and adjust step sizes. This is recommended as the first function to use to solve a problem.

MATLAB's **ode113** function is a multistep solver useful for computationally intensive ODE functions.

Example of ODE solver: `ode45`

The functions are generally called in the same way

```
[t, y] = ode45(odefun, tspan, y0)
```

y: solution array, where each column represents one of the variables and each row corresponds to a time in the *t* vector

odefun: function returning a column vector of the right-hand-sides of the ODEs

tspan: time over which to solve the system

If *tspan* has two entries, the results are reported for those times as well as several intermediate times based on the steps taken by the algorithm

If *tspan* has more than two entries, the results are reported only for those specific times

y0: vector of initial values

Solving a system of ODE using `ode45`

Predator-prey problem

$$\text{Solve } \frac{dy_1}{dt} = 1.2y_1 - 0.6y_1y_2 \quad \frac{dy_2}{dt} = -0.8y_2 + 0.3y_1y_2$$

with $y_1(0)=2$ and $y_2(0)=1$ for 20 seconds

```
function yp = predprey(t, y)
yp = [1.2*y(1)-0.6*y(1)*y(2);...
      -0.8*y(2)+0.3*y(1)*y(2)];
```

`predprey.m` file

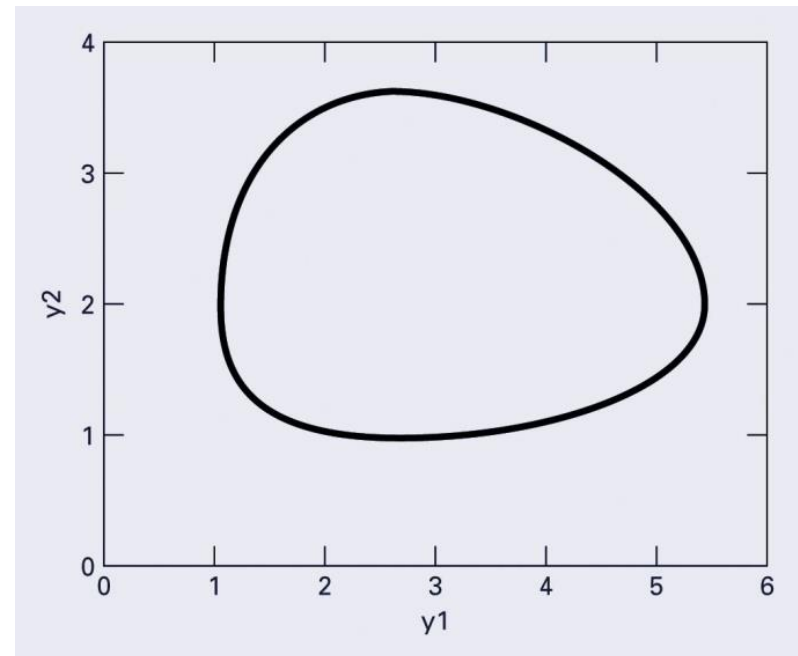
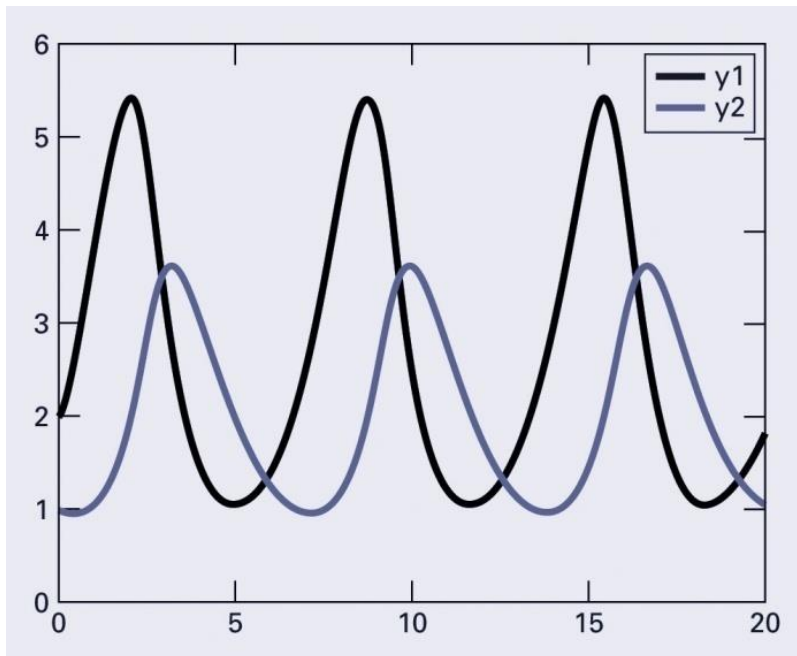
```
tspan = [0 20];
y0 = [2, 1];
[t, y] = ode45(@predprey, tspan, y0);
figure(1); plot(t,y);
figure(2); plot(y(:,1),y(:,2));
```

Example using `ode45`

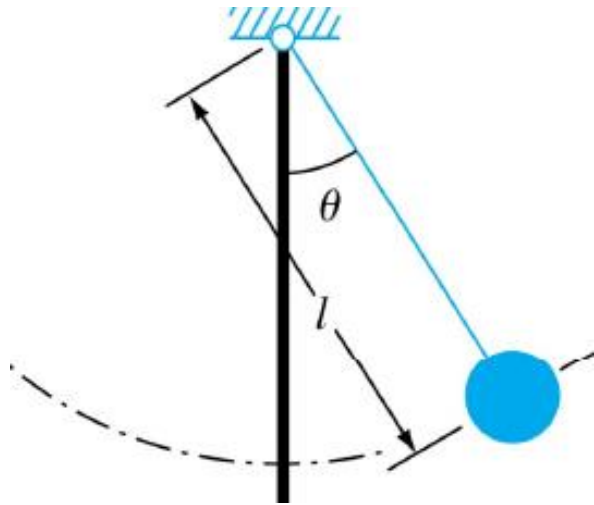
Predator-prey problem

$$\text{Solve } \frac{dy_1}{dt} = 1.2y_1 - 0.6y_1y_2 \quad \frac{dy_2}{dt} = 0.8y_2 + 0.3y_1y_2$$

with $y_1(0)=2$ and $y_2(0)=1$ for 20 seconds



Practice Problem: motion of a pendulum



$$\frac{d^2\theta}{dt^2} + \frac{g}{l} \sin \theta = 0$$

Analytical solution (assuming $\theta = \sin\theta$):

$$\theta(t) = \theta_0 \cos\left(\sqrt{\frac{g}{l}}t\right)$$

If $l = 2$ ft, $g = 32.2$ ft/s² and $\theta_0 = \pi/4$, Solve for θ from $t = 0$ to 1.6 s using

- (a) Euler's method with $h = 0.05$
- (b) Using `ode45` function with $h = 0.05$
- (c) Plot your results and compare with the analytical solution

$$\frac{d^2\theta}{dt^2} + \frac{g}{l} \sin \theta = 0 \quad \Longrightarrow \quad \begin{cases} \frac{d\theta}{dt} = v & @t = 0, \theta_0 = \pi/4 \\ \frac{dv}{dt} = -\frac{g}{l} \sin \theta & @t = 0, v = 0 \end{cases}$$

Boundary value problem: the shooting method

the boundary-value problem is converted into an equivalent initial-value problem.

$$\frac{d^2T}{dx^2} + h'(T_\infty - T) = 0 \Rightarrow \begin{cases} \frac{dT}{dx} = z \\ \frac{dz}{dx} = -h'(T_\infty - T) \end{cases}$$

Generally, the equivalent system will not have sufficient initial conditions

- A guess is made for any undefined values.
- The guesses are changed until the final solution satisfies all the B.C.

For linear ODEs, only two “shots” are required - the proper initial condition can be obtained as a linear interpolation of the two guesses.

Problem: shooting method

Solve
$$\frac{d^2T}{dx^2} + h'(T_\infty - T) + \sigma'(T_\infty^4 - T^4) = 0$$

with $\sigma' = 2.7 \times 10^{-9} \text{ K}^{-3} \text{ m}^{-2}$, $L = 10 \text{ m}$, $h' = 0.05 \text{ m}^{-2}$, $T_\infty = 200 \text{ K}$, $T(0) = 300 \text{ K}$, and $T(10) = 400 \text{ K}$.

First - break into two equations:

$$\frac{d^2T}{dx^2} + h'(T_\infty - T) + \sigma'(T_\infty^4 - T^4) = 0 \Rightarrow \begin{cases} \frac{dT}{dx} = z \\ \frac{dz}{dx} = -0.05(200 - T) - 2.7 \times 10^{-9}(1.6 \times 10^9 - T^4) \end{cases}$$

Problem: shooting method

Code for derivatives:

```
function dy=dydxn(x,y)
dy=[y(2);...
    -0.05*(200-y(1))-2.7e-9*(1.6e9-y(1)^4)];
```

Code for residual:

```
function r=res(za)
[x,y]=ode45(@dydxn, [0 10], [300 za]);
r=y(length(x),1)-400;
```

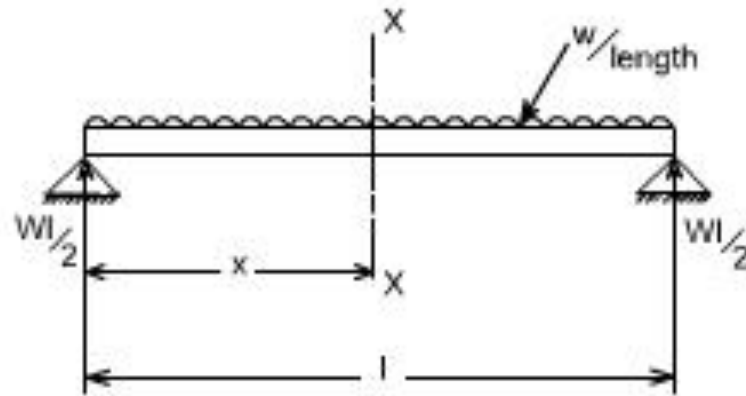
Code for finding root of residual:

```
fzero(@res, -50)
```

Code for solving system:

```
[x,y]=ode45(@dydxn, [0 10], [300 fzero(@res, -50)]);
```

Practice Problem: shooting method



The basic differential equation of the elastic curve for a uniformly loaded beam is given as

$$EI \frac{d^2 y}{dx^2} = \frac{wLx}{2} - \frac{wx^2}{2}$$

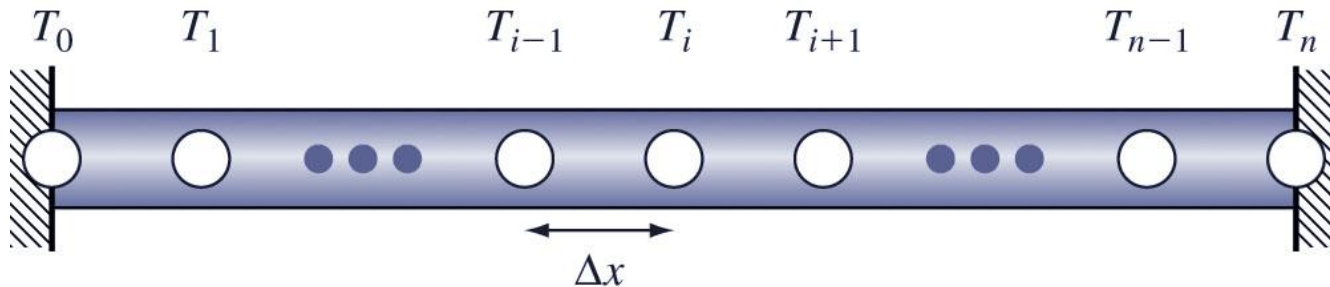
If $E = 30000$ ksi, $I = 800$ in⁴, $w = 1$ kip/ft, $L = 10$ ft, solve for the deflection of the beam using the shooting method and compare the numerical results with the analytical solution

$$y = \frac{wLx^3}{12EI} - \frac{wx^4}{24EI} - \frac{wL^3x}{24EI}$$

Numerical solution: finite difference

finite differences are substituted for the derivatives in the original equation

$$\frac{d^2 T}{dx^2} + h'(T_\infty - T) = 0$$



$$\frac{d^2 T}{dx^2} = \frac{T_{i-1} - 2T_i + T_{i+1}}{\Delta x^2}$$

$$\frac{T_{i-1} - 2T_i + T_{i+1}}{\Delta x^2} + h'(T_\infty - T_i) = 0$$

$$-T_{i-1} + (2 + h'\Delta x^2)T_i - T_{i+1} = h'\Delta x^2 T_\infty$$

Numerical solution: finite difference

The linear differential equation is transformed into a set of simultaneous algebraic equations.

Since T_0 and T_n are known (Dirichlet boundary conditions), they will be on the right-hand-side of the linear algebra system

$$\begin{bmatrix} 2 + h'\Delta x^2 & -1 & & & \\ -1 & 2 + h'\Delta x^2 & & & \\ & & \ddots & & \\ & & & \ddots & \\ -1 & & & & 2 + h'\Delta x^2 \end{bmatrix} \begin{Bmatrix} T_1 \\ T_2 \\ \vdots \\ T_{n-1} \end{Bmatrix} = \begin{Bmatrix} h'\Delta x^2 T_\infty + T_0 \\ h'\Delta x^2 T_\infty \\ \vdots \\ h'\Delta x^2 T_\infty + T_n \end{Bmatrix}$$

Numerical solution: finite difference

If there is a derivative boundary condition (Neumann B.C.), the centered difference equation is solved at the point and rewriting the system equation accordingly.

For a Neumann condition at T_0 point:

$$\left. \frac{dT}{dx} \right|_0 = \frac{T_1 - T_{-1}}{2\Delta x} \Rightarrow T_{-1} = T_1 - 2\Delta x \left(\left. \frac{dT}{dx} \right|_0 \right)$$

$$-T_{-1} + (2 + h'\Delta x^2)T_0 - T_1 = h'\Delta x^2 T_\infty$$

$$-\left[T_1 - 2\Delta x \left(\left. \frac{dT}{dx} \right|_0 \right) \right] + (2 + h'\Delta x^2)T_0 - T_1 = h'\Delta x^2 T_\infty$$

$$(2 + h'\Delta x^2)T_0 - 2T_1 = h'\Delta x^2 T_\infty - 2\Delta x \left(\left. \frac{dT}{dx} \right|_0 \right)$$

Practice Problem: Finite difference

Solve the nondimensionalized ODE using finite difference methods that describe the temperature distribution in a circular rod with internal heat source S

$$\frac{d^2T}{dr^2} + \frac{1}{r} \frac{dT}{dr} + S = 0$$

Over the range $0 \leq r \leq 1$, with boundary conditions

$$T(r=1) = 1 \qquad \left. \frac{dT}{dr} \right|_{r=0} = 0$$

For $S = 1, 10$ and 20 K/m^2 plot the temperature versus radius